Risk-Based Test Strategy and Coverage Plan for an Insurance Claims Platform

1. Objective

The platform has three major quality gaps:

- 50% of production incidents relate to claims calculation and integration failures.
- Product modules and third-party integrations lack adequate automation.
- Test coverage, release readiness, and quality metrics are not tracked consistently across eight Scrum teams.

My approach is to prioritize business-critical risks, extend existing automation, introduce clear release gates, and establish consistent quality metrics across teams.

2. Risk-Based Testing Strategy

I would introduce a risk-scoring model and discuss with teams:

Risk Score = Business Impact × Likelihood of Failure

Both factors are scored from 1–5. Business impact includes financial loss, customer impact, regulatory exposure, and operational disruption.

Score	Risk	Testing approach
20–25	Critical	Extensive testing; mandatory release gate
12–19	High	Automated regression and negative testing
6–11	Medium	Functional and impact-based regression
1–5	Low	Focused testing based on the change

These thresholds are initial guidelines and should be calibrated against historical Critical incidents , critical security or regulatory failures regardless of score.

Implementation:

- Review the last 6–12 months of production incidents.
- Map recurring failures to business rules, services, and integrations.
- Maintain a risk register linking each risk to test scenarios, owners, and release.
- Reassess risk whenever code, business rules, or dependencies change.

3. Test Coverage Strategy

Focus on the areas responsible for the highest business risk:

1. **Assess existing automation:** Review coverage and reliability of existing platform-service tests and identify gaps in product modules and third-party integrations.
2. **Document manual regression:** Create a central inventory of existing test cases, mapped to requirements, business workflows, and risks.
3. **Prioritize high-risk scenarios:** Focus first on incorrect claim payouts, duplicate payments, and integration failures, which account for 50% of production incidents.
4. **Expand automation incrementally:** Automate critical claims calculations and workflows first, followed by third-party API contracts, timeouts, retries, and recovery.
5. **Measure coverage:** Track critical scenarios identified, automated, executed, passed, failed, and not yet covered.

For example, if 100 critical scenarios are identified but only 30 are automated and verified, report the gap transparently rather than treating existing platform automation as complete coverage.

The goal is to move from unknown manual coverage to measurable, risk-based coverage, progressively automating scenarios that provide the greatest reduction in production risk.

4. Automation Architecture and CI/CD

First, assess the existing framework and extend it rather than introducing new tools.

Layer	Coverage	Owners	Suggested tools
Unit	Calculation rules and edge cases	Devs	JUnit, TestNG
API	Claims APIs and business rules	QA	Playwright, Rest Assured
Contract	API compatibility and external services	QA	Pact or schema checks
UI/E2E	Critical customer and agent journeys	QA	Playwright, Selenium
Performance	Critical API throughput and latency	QA	Jmeter, Neoload, k6

Build reusable components for authentication, API clients, test-data setup, assertions, environment configuration, and reporting.

CI/CD gates:

- Pull request: Unit tests by Devs.
- Deployment: API smoke tests & specific service tests.
- Integration environment: Test cross-service workflows, retries, failure recovery, idempotency.
- Pre-release: Critical scenarios and regression selected by risk and code impact.

Run mandatory critical tests for every release. Major releases should include broader cross-product regression and additional performance, security, and recovery testing.

Critical failures block release unless an authorized, documented risk-acceptance process applies.

5. Entry and Exit Criteria

Entry criteria — Testing can start when:

- Requirements and acceptance criteria are reviewed and approved.
- Required test data, policy configurations, and user accounts are available.
- Dependencies are accessible or suitable simulators are ready.
- Unit tests and relevant code-level checks pass.
- Test scenarios are prioritized by risk and linked to requirements.
- Once a deployable build is available, execution can begin.

If a dependency is unavailable, document the limitation and agree on an alternative test approach.

Exit criteria — Testing can be signed off when:

- **Critical scenarios:** 100% executed and passing.
- **High-risk scenarios:** Executed, with failures resolved or formally risk-accepted.
- **Claims calculations:** Expected payouts, limits, rounding and boundary cases pass.
- **Integrations:** Contract, timeout, retry, and duplicate-request scenarios pass.
- **Regression:** Mandatory tests pass; failures are investigated and documented.
- **Defects:** No unresolved critical defects; high-severity defects require documented risk acceptance.

- **Reporting:** Results, coverage gaps, open defects, and residual risks are shared.
- **Automation:** Critical release-gate tests provide reliable results; flaky tests are investigated and cannot be used as evidence of a passing gate.

Release decision: The QA lead recommends readiness based on evidence; the designated release owner makes the final decision. Unresolved critical payout, duplicate-payment, security, or regulatory risks must not be bypassed through informal approval.

Any exception must document the risk, mitigation, business justification, accountable approver, and review date.

6. AI-Assisted Quality Engineering

Use approved AI coding assistants and agent workflows to accelerate test engineering.

Practical use cases:

- Generate test scenarios from requirements and acceptance criteria.
- Identify missing boundary and negative cases.
- Draft API and UI automation code.
- Suggest regression tests based on code changes and dependencies.
- Summarize failures and group similar defects.

Review and execute AI-generated tests. Validate expected payouts against approved business rules; never rely on unverified AI-generated expected results.

7. Metrics and Quality Dashboard

Use existing test management and reporting tools to provide a shared view across all teams.

Metric	**Initial target**
Critical-risk coverage	100% critical scenarios passing before release
Claims incidents	Reduce by 30% within 90 days, subject to baseline validation
Regression time	Reduce by 50% from the measured baseline
Automation coverage	Increase coverage of agreed critical scenarios every sprint
Flaky tests	Below 2%, with ownership and remediation

Release quality	Track production defects, rollback rate, and hot fix frequency

Review metrics every sprint. Automation percentage alone is not proof of quality; production outcomes and defect trends determine whether the strategy is effective.

8. Team Ownership

Establish common standards while keeping quality ownership within the teams.

- **Three product teams:** Product rules, product APIs, and product regression.
- **Three platform teams:** Shared Service testing and reusable automation components.
- **Two integration teams:** Contract testing, external dependencies, and recovery scenarios.
- **All developers and QA engineers:** Automation, code-level checks, and defect prevention.
- **Quality Engineering Working Group:** Shared standards, cross-team risks, and consolidated metrics.

Each team owns the quality of its changes. The definition of done includes relevant automated tests, updated risk assessments, and no unresolved critical defects without approved risk acceptance.

9. 90-Day Implementation Roadmap

Timeline	Deliverables
Weeks 1–2	Review incidents, map critical workflows, assess automation, establish baseline metrics
Weeks 3–4	Automate priority calculation scenarios, add contract tests, introduce initial CI/CD gates
Weeks 5–8	Expand automation, cover failure recovery, publish a shared dashboard
Weeks 9–12	Roll out reusable components, risk-based regression, measure improvements

Start with one high-risk claims workflow and one critical third-party integration. Validate the approach, then reuse the framework, pipeline checks, and reporting across all eight teams.

AI can accelerate implementation, but engineering review, deterministic validation, and clear release controls remain mandatory.

10. Expected Outcomes

- Fewer incorrect payouts and integration-related production incidents.
- Earlier detection of API incompatibility and service failures.
- Reduced manual regression effort and faster feedback.
- Consistent quality standards and ownership across eight teams.
- Clear, evidence-based release decisions and visibility into remaining risks.

My focus as Technical Test Architect would be to protect critical business workflows, build reusable automation, integrate quality gates into CI/CD, and use production data to continuously improve testing priorities.